Detekcja deepfake’ów

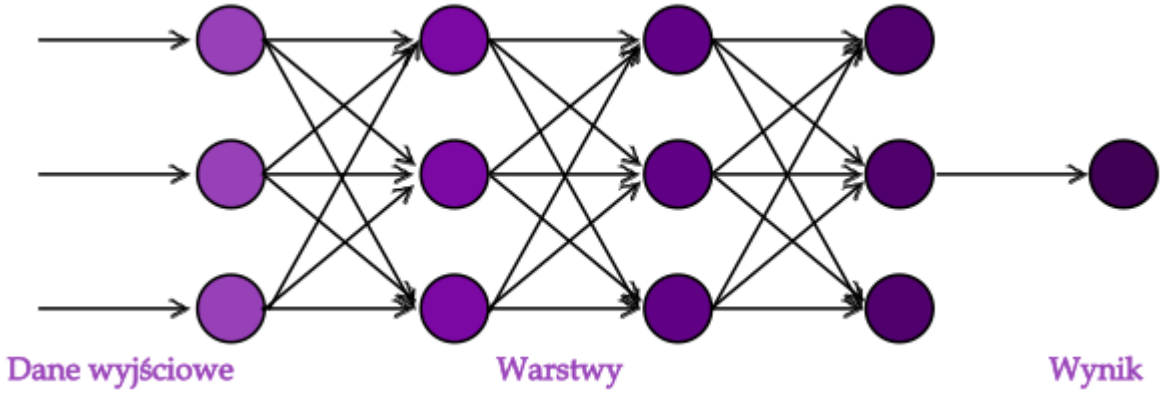
Autor wiodący: Krzysztof Grabias

Autor wspomagający: Chat GPT-v. 3.5

Redakcja: Krzysztof Grabias

1 Budowa modelu, trenowanie sieci neuronowej

„*Sieć neuronowa – system przeznaczony do przetwarzania informacji, którego budowa i zasada działania są w pewnym stopniu wzorowane na funkcjonowaniu fragmentów rzeczywistego (biologicznego) systemu nerwowego.*” - definicja sieci neuronowej
Źródło: https://pl.wikipedia.org/wiki/Sieć_neuronowa Dostęp: 22.05.2024



Schemat prostej sieci neuronowej. Źródło: opracowanie własne

„*W kontekście sieci neuronowych, termin "model" odnosi się do matematycznego lub obliczeniowego opisu struktury sieci neuronowej, która została przeszkolona na zbiorze danych w celu wykonywania określonych zadań, takich jak klasyfikacja, regresja, segmentacja obrazu itp.*”
- definicja modelu Źródło: Chat GPT-4o

Do zbudowania modelu użyto przekształcony kod Diego Guizan’a Lopez’a zamieszczony na forum internetowym www.kaggle.com/code/diegoguizanlopez/it-s-an-ai-or-a-person, który wykorzystuje Keras – część biblioteki Tensorflow zaprojektowanej przez Google Brain Team. Uruchomiono go w Pythonie w środowisku Pycharm z oprogramowaniem Jupyter, które pozwala na uruchamianie części kodu w postaci bloków. Model wytrenowano na zbiorze zdjęć udostępnionym również na tym forum (można ten zbiór znaleźć pod w.w. linkiem), uzupełnionym własnymi zdjęciami oraz fałszywymi zdjęciami wygenerowanymi za pomocą strony thispersondoesnotexist.com.

1.1 Importowanie bibliotek

```
import tensorflow as tf
import keras
from keras import layers, models
import os
import numpy as np
from matplotlib import pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

Tensorflow, Keras – biblioteki służące do tworzenia sieci neuronowych
os – biblioteka służąca m.in. do manipulowania plikami i katalogami
numpy – biblioteka służąca do obliczeń naukowych
matplotlib – biblioteka służąca do tworzenia wykresów
scikit-learn – biblioteka łącząca wybrane funkcje wszystkich powyższych

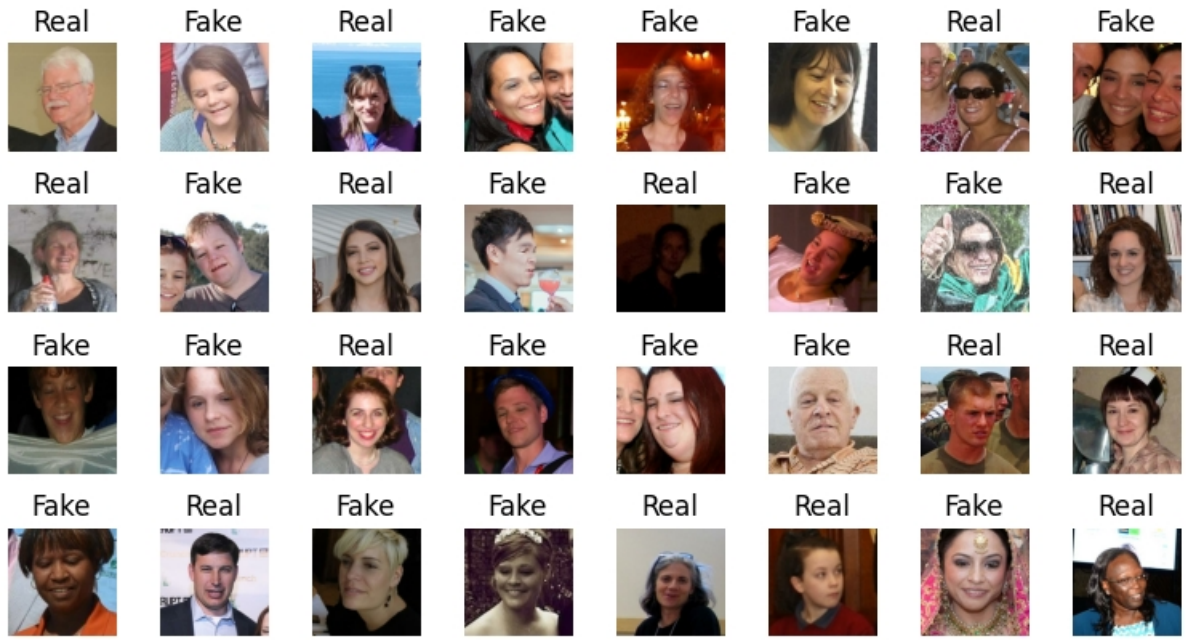
1.2 Wybranie danych danych

```
path = "C:\\Users\\Krzychu\\Desktop\\IS\\Dataset"
def get_from_dir(dir):
    global path
    dir = os.path.join(path,dir)
    return tf.keras.utils.image_dataset_from_directory(
            dir,
            labels='inferred',
            color_mode="rgb",
            seed=42,
            batch_size=32,
            image_size=(128, 128))
train = get_from_dir("Train")
test = get_from_dir("Test")
val = get_from_dir("Validation")
```

Do zmiennej „path” przypisano ścieżkę do katalogu zawierającego zbiór zdjęć prawdziwych i fałszywych. Następnie wszystkim zdjęcia przeskalowano do 128x128px oraz podzielono zdjęcia na 32-elementowe partie (batch_size). „seed” to dowolna liczba, która decyduje o kolejności wybierania zdjęć do partii.

Narysowano wykres przedstawiający przykładową partię (batch):

```
class_names = np.unique(train.class_names) #pobiera nazwy klas (folderów) z folderu Train
#wykres
n_rows = 4
n_cols = 8
plt.figure(figsize=(n_cols * 1.2, n_rows * 1.2))
data = list(train.take(n_rows*n_cols))
for row in range(n_rows):
    for col in range(n_cols):
        index = n_cols * row + col
        single_image = data[index][0][0]
        plt.subplot(n_rows, n_cols, index + 1)
        plt.imshow(single_image.numpy().astype("uint8"),cmap="gray")
        plt.axis('off')
        label_index = data[index][1][0]
        plt.title(class_names[label_index], fontsize=12)
plt.subplots_adjust(wspace=0.2, hspace=0.5)
plt.show()
```



1.3 Budowa modelu

```
model = models.Sequential()

model.add(layers.Conv2D(32,(3,3),activation="relu", padding='same',input_shape=(128,128,3)))
model.add(layers.Conv2D(32,(3,3),activation="relu", padding='same'))
model.add(layers.MaxPooling2D((2,2), strides=(2, 2)))

model.add(layers.Conv2D(64,(3,3),activation="relu", padding='same'))
model.add(layers.Conv2D(64,(3,3),activation="relu", padding='same'))
model.add(layers.MaxPooling2D((2,2), strides=(2, 2)))

model.add(layers.Conv2D(128,(3,3),activation="relu", padding='same'))
model.add(layers.Conv2D(128,(3,3),activation="relu", padding='same'))
model.add(layers.MaxPooling2D((2,2), strides=(2, 2)))

model.add(layers.Flatten())
model.add(layers.Dropout(0.5))
model.add(layers.Dense(128,activation="relu"))
model.add(layers.Dense(256,activation="relu"))
model.add(layers.Dense(1,activation="sigmoid"))

model.summary()
```

Utworzenie sekwencyjnego modelu sieci neuronowej.

1. Dodanie warstwy konwolucyjnej 2D z 32 filtrami, aktywacją ReLU, wypełnieniem 'same' i określeniem kształtu wejściowego.
2. Dodanie kolejnej warstwy konwolucyjnej 2D z 32 filtrami, aktywacją ReLU i wypełnieniem 'same'.
3. Dodanie warstwy maksymalnego spoolingowania z rozmiarem okna 2x2 i krokiem 2x2.
4. Dodanie warstwy konwolucyjnej 2D z 64 filtrami, aktywacją ReLU i wypełnieniem 'same'.
5. Dodanie kolejnej warstwy konwolucyjnej 2D z 64 filtrami, aktywacją ReLU i wypełnieniem 'same'.
6. Dodanie warstwy maksymalnego spoolingowania z rozmiarem okna 2x2 i krokiem 2x2.
7. Dodanie warstwy konwolucyjnej 2D z 128 filtrami, aktywacją ReLU i wypełnieniem 'same'.
8. Dodanie kolejnej warstwy konwolucyjnej 2D z 128 filtrami, aktywacją ReLU i wypełnieniem 'same'.
9. Dodanie warstwy maksymalnego spoolingowania z rozmiarem okna 2x2 i krokiem 2x2.
10. Spłaszczenie danych wejściowych do jednowymiarowego wektora.
11. Dodanie warstwy dropout z prawdopodobieństwem odrzucenia 50% połączeń.
12. Dodanie w pełni połączonej warstwy z 128 neuronami i aktywacją ReLU.
13. Dodanie w pełni połączonej warstwy z 256 neuronami i aktywacją ReLU.
14. Dodanie warstwy wyjściowej z 1 neuronem i aktywacją sigmoid.
15. Wyświetlenie podsumowania modelu, pokazujące architekturę, liczbę parametrów i kształty warstw.

Objaśnienie wykonanych kroków sporządzone przez Chat GPT-3.5.

1.4 Kompilacja modelu

```
model.compile('adam',loss=tf.keras.losses.BinaryCrossentropy(), metrics=['accuracy'])
early_stopping_cb = keras.callbacks.EarlyStopping(patience=5,restore_best_weights=True)
history = model.fit(train,epochs=16,batch_size=32,validation_data=val,callbacks=[early_stopping_cb])
model.evaluate(test)
model.save("moj_model.keras")
```

model.compile('adam',loss=tf.keras.losses.BinaryCrossentropy(), metrics=['accuracy']):
„adam" – stochastyczny algorytm optymalizacji treningu. Pełna nazwa to „Adaptive Moment Estimation optimizer"
early_stopping_cb=keras.callbacks.EarlyStopping(patience=5,restore_best_weights= True) -
Zatrzymywanie procesu treningu gdy skuteczność przestaje się poprawiać

history=model.fit(train,epochs=16,batch_size=32,validation_data=val,callbacks=[early_stopping_cb]):
train – Zbiór zdjęć treningowych.
epochs=16 - Liczba epok treningowych. Model był trenowany przez 20 pełnych przejść przez cały zbiór danych treningowych.
batch_size=32 – Rozmiar partii. Zbiór danych treningowych został podzielony na partie po 32 próbki, a model był aktualizowany po przetworzeniu każdej partii.
validation_data=val - Zbiór danych porównawczych, używany do oceny modelu po każdej epoce.
model.evaluate(test) – ocenienie skuteczności modelu na podstawie zdjęć z katalogu test
model.save(„moj_model2.keras") – zapisanie modelu

Kompilowanie modelu trwało kilkanaście godzin, raport z kompilacji można znaleźć w pliku trening_modelu2.txt

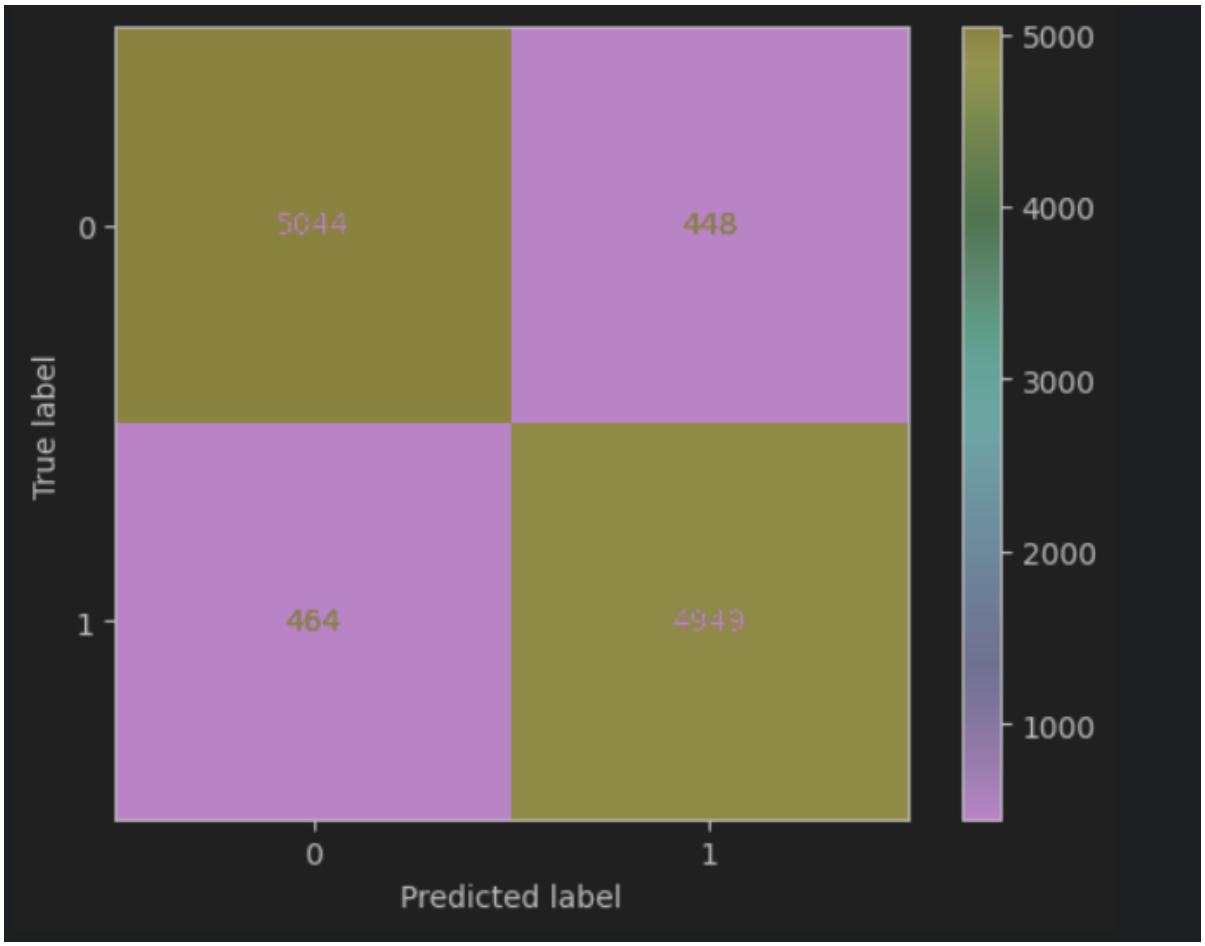
1.5 Narysowanie macierzy błędów (ang. confussion matrix)

```
true_labels = []
predictions = []

for images, labels in test:
    true_labels.extend(labels.numpy())
    batch_predictions = model.predict(images) >= 0.5
    predictions.extend(batch_predictions.astype(int).flatten())

true_labels = np.array(true_labels)
predictions = np.array(predictions)

cm = confusion_matrix(true_labels, predictions)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
plt.show()
```



Wynika z niej, że skuteczność modelu wynosi 95,2%.

2 Instrukcja obsługi aplikacji

Aplikację obsługującą model zaleca się uruchamiać w środowisku Pycharm. Należy stworzyć nowy projekt, a następnie dodać do niego pliki z katalogu Detektor_deepfake, w szczególności main.py oraz „moj_model2.keras”. Aby móc uruchomić aplikację należy zainstalować wymagane biblioteki: sys, PyQt5, numpy, Keras, Tensorflow. Biblioteka sys powinna być zainstalowana domyślnie. Najłatwiej to zrobić używając polecenia pip install lub pip3 install w terminalu.
Przykład:

Można pobrać wszystkie w.w. biblioteki jednym poleceniem w terminalu:

pip install numpy, pyqt5, keras, tensorflow

Sprawdzić czy biblioteki są już zainstalowane można za pomocą pip show
Przykład:

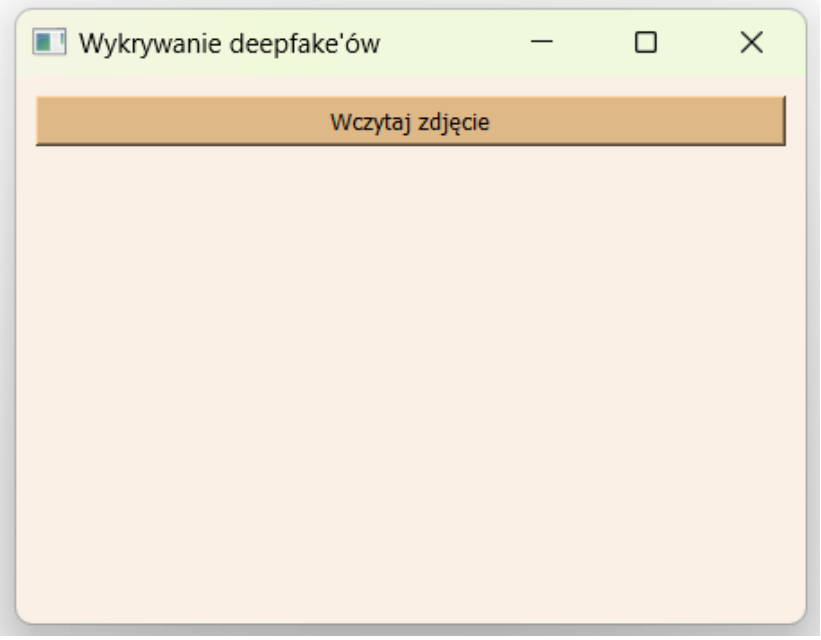
```
(venv) PS C:\Users\Krzychu\PycharmProjects\SIv3> pip show opencv-python
Name: opencv-python
Version: 4.9.0.80
Summary: Wrapper package for OpenCV python bindings.
Home-page: https://github.com/opencv/opencv-python
Author:
Author-email:
License: Apache 2.0
Location: C:\Users\Krzychu\PycharmProjects\SIv3\venv\Lib\site-packages
Requires: numpy, numpy, numpy, numpy, numpy, numpy
Required-by:
(venv) PS C:\Users\Krzychu\PycharmProjects\SIv3>
```

Opcjonalnie można zmienić ścieżkę modelu w pierwszej linijce kodu. Do projektu załączono trzy modele: „moj_model.keras2”, „moj_model.keras”, „model_z_neta.keras” (wymienione według skuteczności).

```
1   model_path="moj_model.keras"
```

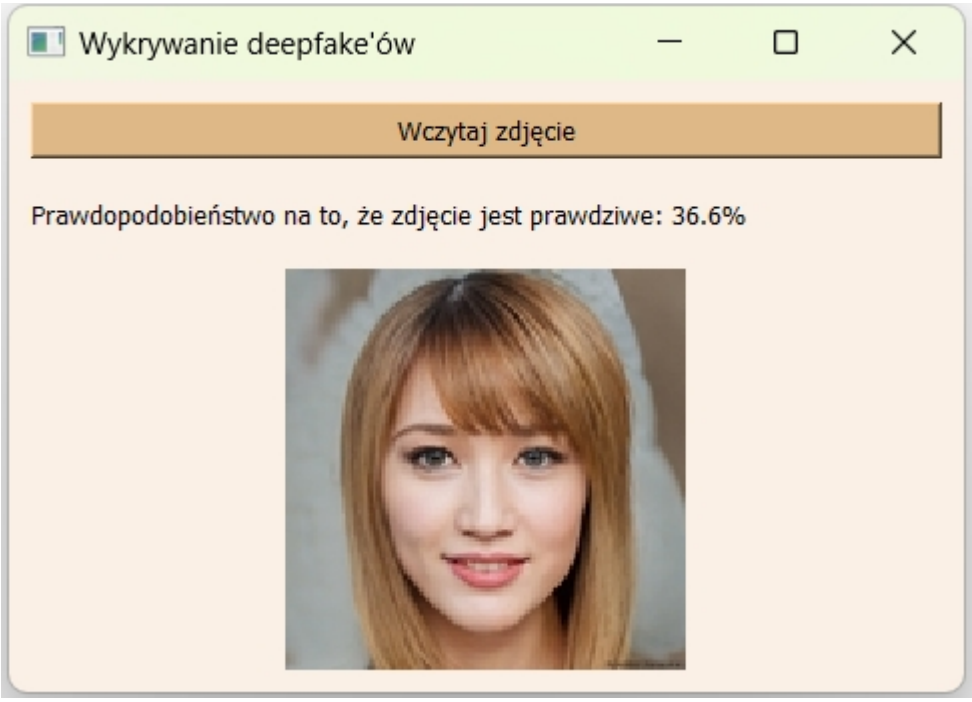
Następnie należy program uruchomić klikając zieloną strzałkę znajdującą się na górnym pasku Pycharm’a lub używając kombinacji klawiszowej Shift+F10.

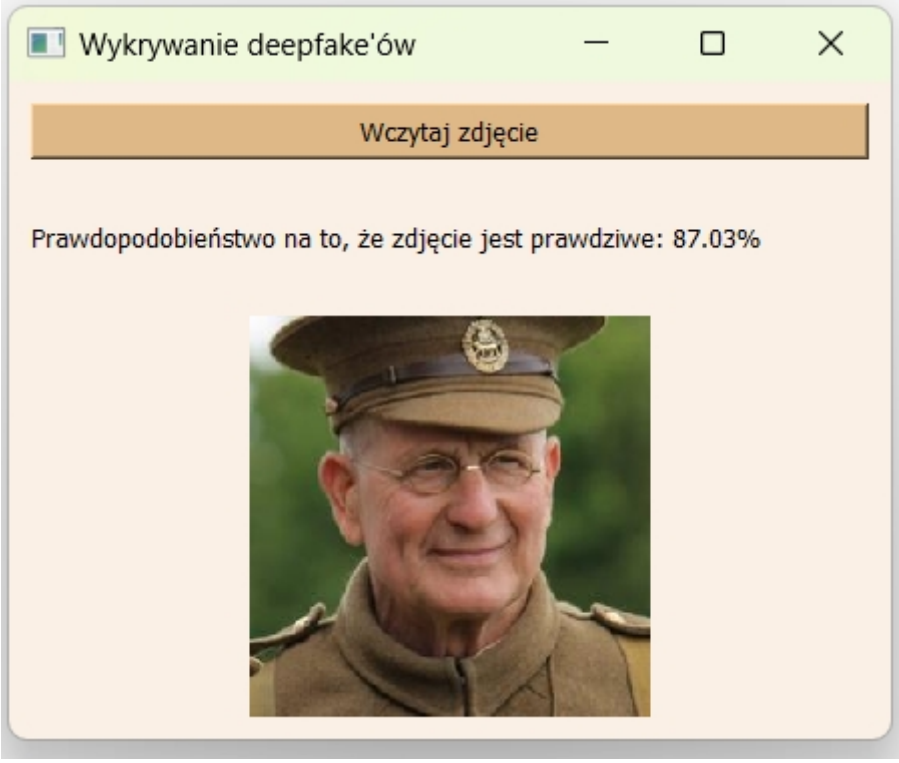
Po chwili pojawi się okno interfejsu graficznego. Pojawiające się w konsoli ostrzeżenia biblioteki Tensorflow należy zignorować.

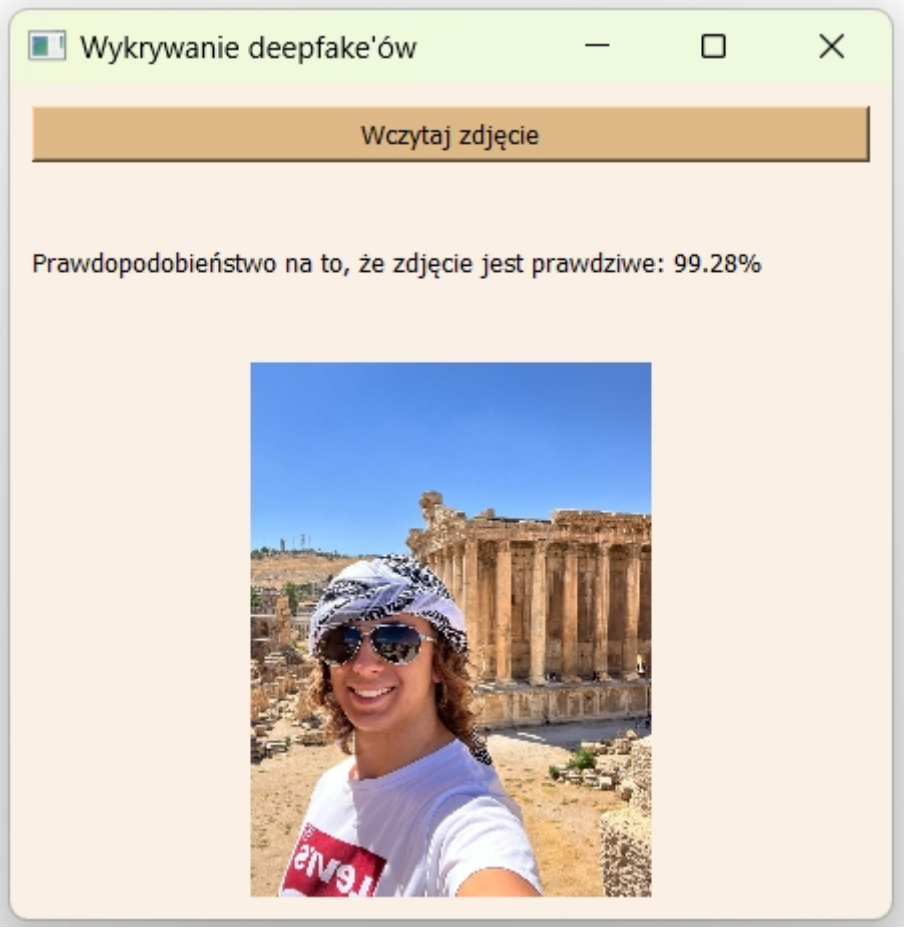


Następnie należy kliknąć przycisk podpisany „Wczytaj zdjęcie”. Otworzy się okno dialogowe, za pomocą którego należy wybrać zdjęcie twarzy.
Po wybraniu obrazu ukażą się szacunkowe szanse na jego autentyczność.

Przykładowe widoki okna z wybranym zdjęciem:







Po odczytaniu wyniku można sprawdzić następne zdjęcie ponownie je wybierając za pomocą przycisku podpisanego „Wczytaj zdjęcie”.

3 Przykładowe ścieżki rozwoju aplikacji

3.1 Obsługa wielu zdjęć w jednym momencie

Bardzo prosto można zmienić kod tak, aby aplikacja obsługiwała jednorazowo więcej niż jedno zdjęcie. Następnie należy zaprojektować nowe GUI lub zapisać wyniki do pliku .csv (np. za pomocą biblioteki Pandas).

3.2 Automatyczne wycinanie zdjęć twarzy

Dobrym kierunkiem rozwoju jest obsługa zdjęć grupowych lub zdjęć całych sylwetek ludzkich za pomocą automatycznego wycinania twarzy.
Przykładowa funkcja:

```
def detect_faces(image):
    face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))
    detected_faces = []
    for (x, y, w, h) in faces:
        face = image[y:y+h, x:x+w]
        detected_faces.append(face)
    return detected_faces
```

3.3 „Spakowanie” aplikacji do pliku .exe

Żadna komercjalna aplikacja nie przebiłaby się na rynku gdyby do jej użycia wymagana by była instalacja wybranego środowiska programistycznego oraz wielu pakietów za pomocą terminala. Dobrym pomysłem jest przekształcenie aplikacji konsolowej na aplikację uruchamianą bezpośrednio z pliku .exe.